



**CAMPUS
UNIVERSITÁRIO DE
SAQUAREMA**

UNIVASSOURAS

CAMPUS UNIVERSITÁRIO DE SAQUAREMA

CURSO DE ENGENHARIA DE SOFTWARE

ÁRVORES EM ESTRUTURA DE DADOS

Saquarema

2025



Autores:

202422712 - Bruno Goulart dos Santos

202422999 - Bruno Pires

202423468 - Diego Cosme de Souza

202422892 - Eduardo Rodinei Irineu da Silva

202411292 - Felipe Casagrande Oliveira

202422989 - Josias Andrade

202422981 - Lucas Borba

202422748 - Pedro Lucas Santana Motta

202322988 - Renata Mendes

202422974 - Thiago Moura

Trabalho apresentado à disciplina
Estrutura de Dados,

do curso de Engenharia de Software da
Univassouras,

como requisito parcial para a avaliação
semestral.

Professor: André Saraiva

Saquarema, 2025

Sumário

1. Definição.....	4
2. Construção	4
2.1 Primeiros passos para a construção de uma árvore binária	4
3. Percorrendo Árvores Binárias	5
4. Operações	6
4.1 Inserir	6
4.2 Buscar	6
4.3 Atualizar.....	7
4.4 Excluir.....	7
5. Árvores AVL.....	8
6. Árvores Rubro-Negras.....	10
7. Árvores Digitais (Tries)	12
8. Árvores de Huffman	13
9. PDL – Árvore B	16
10. Árvores Balanceadas	18
11. Conclusão.....	19
12. Quadro comparativo.....	20
13. Referências.....	22

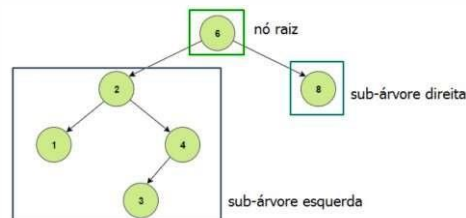
Introdução

As árvores são estruturas de dados hierárquicas amplamente utilizadas em Ciência da Computação para organizar informações de maneira eficiente. Cada árvore é composta por nós, conectados de forma que exista um nó raiz e, a partir dele, ramos que se subdividem em nós filhos. Entre os principais tipos, destacam-se as árvores binárias, nas quais cada nó possui até dois filhos, as árvores balanceadas, como AVL e Rubro-Negras, que mantêm equilíbrio para otimizar operações, e as árvores digitais (tries), voltadas para armazenamento e busca de sequências de caracteres. A importância do estudo das árvores está na sua capacidade de melhorar o desempenho em operações de inserção, busca e remoção, reduzindo a complexidade em relação a estruturas lineares. Suas aplicações incluem sistemas de arquivos, indexação em bancos de dados, compressão de dados e algoritmos de inteligência artificial. Portanto, o conhecimento sobre árvores é essencial para a formação em Estruturas de Dados.

Palavras-chave: Árvores. Estruturas de Dados. Algoritmos. Busca.

1. Definição

Uma árvore binária é uma estrutura de dados composta por nós, capaz de armazenar uma quantidade finita de elementos. Esses nós são organizados de forma hierárquica, sem a formação de ciclos ou caminhos fechados. Cada nó pode conter um valor e estar associado a, no máximo, duas subárvores: uma à esquerda e outra à direita. Todos os nós da subárvore à esquerda possuem valores menores que o do nó raiz, enquanto os da subárvore à direita armazenam valores maiores. Os nós que não possuem filhos são denominados folhas.



Os nós contêm todos os elementos armazenados na árvore. A raiz é o elemento localizado no topo da árvore. O nó pai é aquele que se encontra acima de um nó filho e está diretamente ligado a ele. Um nó pai pode ter, no máximo, dois filhos. O grau de um nó corresponde ao número de filhos que ele possui.

Outras propriedades:

- **Árvore binária estrita:** todo nó não folha possui obrigatoriamente dois filhos.
- **Árvore completa:** todas as folhas encontram-se no mesmo nível.
- **Árvore cheia:** todas as folhas encontram-se no nível d ou no nível $d-1$.

2. Construção

2.1 Primeiros passos para a construção de uma árvore binária:

Para representarmos um elemento da árvore (um nó), será definido um registro da seguinte maneira:

```

typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
} Node;
  
```

Para criar um novo nó com valor “data”:

```
Node* createNode(int data) {  
    Node* newNode = (Node*)malloc(sizeof(Node));  
    newNode->data = data;  
    newNode->left = NULL;  
    newNode->right = NULL;  
    return newNode;  
}
```

Acima, os ponteiros da esquerda e direita são inicializados como NULL.

```
void printTree(Node* root, int space) {  
    if (root == NULL) return;  
    space += 5;  
    printTree(root->right, space);  
    printf("\n");  
    for (int i = 5; i < space; i++) printf(" ");  
    printf("%d\n", root->data);  
    printTree(root->left, space);  
}
```

Para imprimir a árvore de forma visual:

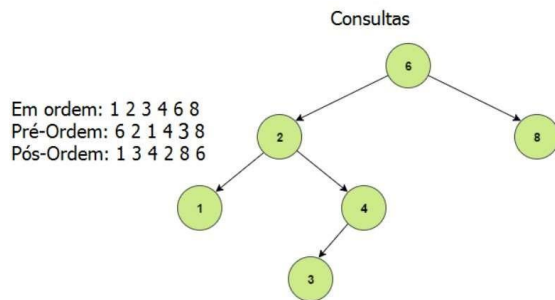
```
typedef struct Node {  
    int data;  
    struct Node* left;  
    struct Node* right;  
} Node;
```

3. Percorrendo Árvores Binárias

Existem três métodos principais:

1. Pré-ordem: raiz → esquerda → direita
2. Em ordem: esquerda → raiz → direita
3. Pós-ordem: esquerda → direita → raiz

[Exemplos de funções em C para cada tipo]



Função para percorrer a árvore em pré-ordem:

```

void preOrder(Node* root) {
    if (root == NULL) return;
    printf("%d ", root->data);
    preOrder(root->left);
    preOrder(root->right);
}
  
```

Função para percorrer a árvore em ordem:

```

void inOrder(Node* root) {
    if (root == NULL) return;
    inOrder(root->left);
    printf("%d ", root->data);
    inOrder(root->right);
}
  
```

Função para percorrer a árvore em pós-ordem:

```

void postOrder(Node* root) {
    if (root == NULL) return;
    postOrder(root->left);
    postOrder(root->right);
    printf("%d ", root->data);
}
  
```

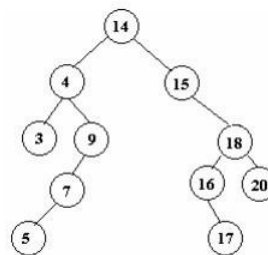
4. Operações

4.1 Inserir (insert)

Inserir um dado específico, mantendo a ordem da árvore.

```

Node* insert(Node* root, int data) {
    if (root == NULL) {
        return createNode(data);
    }
    if (data < root->data) {
        root->left = insert(root->left, data);
    } else {
        root->right = insert(root->right, data);
    }
    return root;
}
  
```



4.2 Buscar (search)

Procurar um dado específico. Se não encontrado, retorna NULL.

```
Node* search(Node* root, int data) {
    if (root == NULL || root->data == data) {
        return root;
    }
    if (data < root->data) {
        return search(root->left, data);
    } else {
        return search(root->right, data);
    }
}
```

4.3 Atualizar (update)

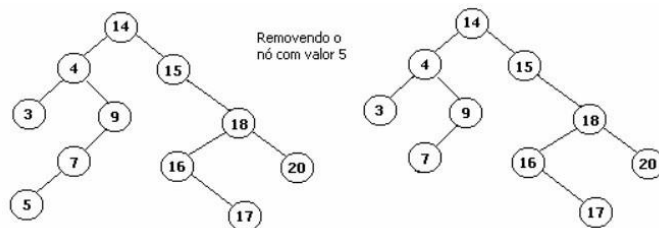
Remove um nó com valor antigo e insere o novo valor.

```
Node* update(Node* root, int oldValue, int newValue) {
    root = deleteNode(root, oldValue);
    root = insert(root, newValue);
    return root;
}
```

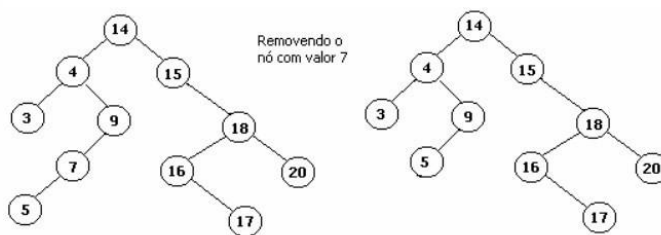
4.4 Excluir (delete)

Três casos:

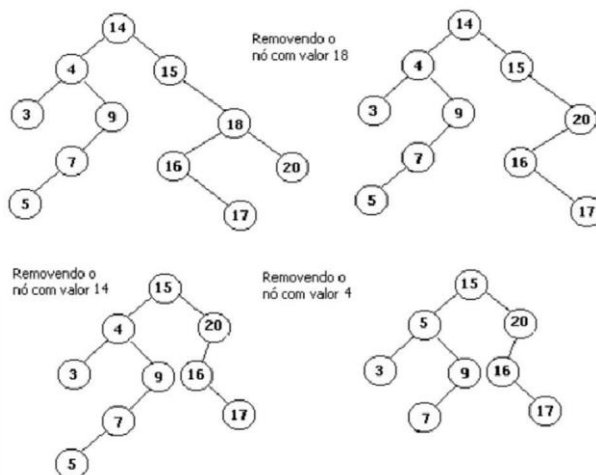
- Nó folha: basta remover.



- Nó com um filho: filho substitui o nó removido.



- Nó com dois filhos: substituir pelo menor valor da subárvore direita.



```
Node* deleteNode(Node* root, int data) {
    if (root == NULL) {
        return root;
    }
    if (data < root->data) {
        root->left = deleteNode(root->left, data);
    } else if (data > root->data) {
        root->right = deleteNode(root->right, data);
    } else {
        // Nodo com apenas um filho ou nenhum filho
        if (root->left == NULL) {
            Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            Node* temp = root->left;
            free(root);
            return temp;
        }
        // Nodo com dois filhos:
        Node* temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}
```

5. Árvores AVL

Introdução

As árvores AVL são um tipo especial de árvore binária de busca (ABB) que possui a capacidade de se manter balanceada automaticamente após inserções e remoções de elementos. Esse tipo de estrutura foi criado em 1962 por Georgy Adelson-Velsky e Evgenii Landis, sendo as iniciais de seus sobrenomes a origem do nome AVL.

O que é uma Árvore Binária de Busca?

Uma árvore binária de busca (ABB) é uma estrutura de dados onde:

- ☐ Cada nó possui no máximo dois filhos (esquerda e direita).
- ☐ Os valores à esquerda de um nó são sempre menores.
- ☐ Os valores à direita de um nó são sempre maiores.

Apesar de eficiente, uma ABB pode ficar desequilibrada, transformando-se em uma estrutura semelhante a uma lista, o que degrada a eficiência da busca.

Definição de Árvore AVL

Uma árvore AVL garante que, para cada nó, a diferença de altura entre a subárvore esquerda e a subárvore direita seja no máximo 1.

O cálculo é feito pelo fator de balanceamento (FB):

$FB = \text{Altura}(\text{Subárvore Esquerda}) - \text{Altura}(\text{Subárvore Direita})$

- ☐ Se $FB = -1, 0, 1 \rightarrow$ a árvore está balanceada.
- ☐ Se $|FB| > 1 \rightarrow$ a árvore está desbalanceada e precisa de rotações.

Rotações na Árvore AVL

Quando ocorre um desbalanceamento, a árvore aplica rotações para restaurar o equilíbrio:

1. Rotação Simples à Direita (RR)

- Corrige desbalanceamento Esquerda-Esquerda.

2. Rotação Simples à Esquerda (LL)

- Corrige desbalanceamento Direita-Direita.

3. Rotação Dupla à Direita (LR)

- Corrige desbalanceamento Esquerda-Direita.

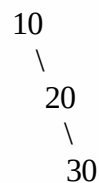
4. Rotação Dupla à Esquerda (RL)

- Corrige desbalanceamento Direita-Esquerda.

Essas operações garantem que a árvore se mantenha sempre eficiente.

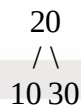
Exemplo:

Inserindo os números na ordem: 10, 20, 30 em uma árvore binária normal, teríamos:



Essa árvore está desequilibrada.

Na AVL, é aplicada uma rotação simples à esquerda, resultando em:



Agora a árvore está balanceada novamente.

Vantagens da Árvore AVL

- Mantém operações de busca, inserção e remoção em $O(\log n)$.
- Sempre próxima de uma árvore perfeitamente balanceada.
- Evita que a estrutura se torne ineficiente como uma lista encadeada.

Desvantagens da Árvore AVL

- Inserção e remoção podem ser mais custosas, devido à necessidade de rotações.
- Implementação mais complexa em comparação com uma árvore binária simples.

Aplicações das Árvores AVL

- Sistemas de bancos de dados para buscas rápidas.
- Compiladores e interpretadores, em tabelas de símbolos.
- Sistemas operacionais, para gerenciamento eficiente de memória.
- Qualquer aplicação que precise de busca rápida e balanceada.

Conclusão

As árvores AVL representam uma das estruturas mais importantes da ciência da computação para garantir eficiência em buscas, inserções e remoções. Seu balanceamento automático por meio de rotações faz com que seja amplamente utilizada em contextos onde o desempenho é crítico.

6. Árvores Rubro-Negras

- Definição e histórico

- É um tipo de árvore binária balanceada
- Originalmente criada por Rudolf Bayer em 1972 e chamadas de "Árvores Binárias Simétricas"
- Posteriormente, em um trabalho de Leonidas J. Guibas e Robert Sedgwick de 1978, ela adquiriu o seu nome atual
- Utiliza um esquema de coloração dos nós para manter o balanceamento da árvore

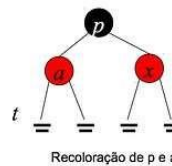
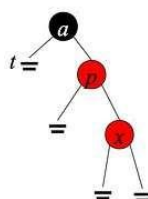
// comentários do professor: Diferente da Arvore avl que usa um esquema de alturas, a arvore rubro negra uso um esquema de coloração dos nós. //

- Nó da árvore possui um atributo de cor que pode ser "**vermelho**" ou "**preto**"

- Propriedades

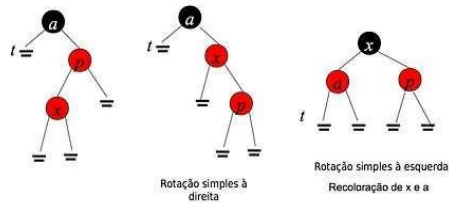
- Todo nó da árvore é "**vermelho**" ou "**preto**"
- A "**raiz**" é sempre "**preta**"
- Todo nó folha ("**NULL**") é "**preto**"
- Se um nó é "**vermelho**", então os seus filhos são "**pretos**" (ou seja, não existem nós "**vermelhos**" consecutivos)
- Para cada nó, todos os caminhos desse nó para os nós folhas descendentes contêm o mesmo número de nós "**pretos**"

- Inserção: casos 1, 2 e 3 (subcasos)



Subcaso 2:

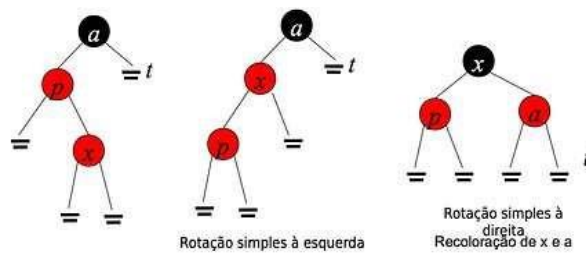
Rotação à esquerda.



Subcaso 3:

Rotação dupla esquerda.

Pode ser visto como um subcaso 1 seguido do subcaso 2.



- Remoção: passos gerais: voltar aqui
- Comparação AVL x Rubro-Negra

7. Árvores Digitais (Tries)

- Definição

As árvores digitais, também chamadas de tries, são estruturas de dados usadas para armazenar coleções de cadeias de caracteres (strings) de forma hierárquica e eficiente. Cada nível da árvore representa um caractere da string, e os caminhos da raiz até os nós folha correspondem às palavras armazenadas. Diferente das árvores binárias, as árvores digitais não comparam chaves inteiras; elas exploram os dígitos ou caracteres de cada chave. São amplamente utilizadas em sistemas de busca, dicionários de palavras, autocompletar e compressão de dados.

- Estrutura de nós e caminhos

- Cada nó representa um caractere ou dígito.
- A raiz representa o início (vazio).
- Cada caminho raiz ® folha forma uma palavra ou chave.
- Nós podem armazenar uma marca indicando o fim de uma palavra.

- Vantagens

Número de ponteiros NULL menor (ocupa menos espaço) {} Chaves podem ser facilmente convertidas em binário

-Desvantagens

- Podem consumir muita memória para alfabetos grandes.
- Estrutura mais complexa em comparação a arrays ou listas simples.

- Aplicações práticas

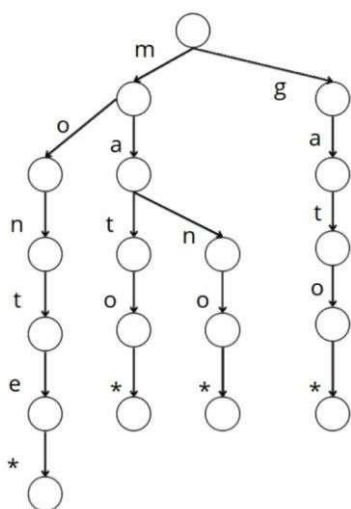
Uso em Algoritmos:

- Em sistemas de busca, permite localizar rapidamente palavras por prefixos.
- Em compressão (como em algoritmos de Huffman modificados), ajuda a reduzir redundâncias.
- Em redes, é usada para roteamento de IPs (trie binária).

Estrutura de uma Árvore Digital:

- Cada nó representa um caractere ou dígito.
- A raiz representa o início (vazio).
- Cada caminho raiz ® folha forma uma palavra ou chave.
- Nós podem armazenar uma marca indicando o fim de uma palavra.

Figura : Exemplo de Árvore Digital



Considerações Finais sobre Tries

As Tries são uma ferramenta poderosa no arsenal de estruturas de dados, especialmente para aplicações que envolvem manipulação de strings. Sua capacidade de armazenar palavras de forma eficiente e realizar buscas rápidas as torna ideais para sistemas de busca, dicionários e autocompletar. No entanto, como qualquer estrutura de dados, é importante considerar suas limitações e avaliar se a Trie é a melhor escolha para o problema em questão. Compreender as nuances da Trie pode ajudar desenvolvedores a criar soluções mais eficientes e eficazes em seus projetos.

8. Árvores de Huffman em C

Objetivo: compressão de dados

A compressão de dados é um pilar da ciência da computação moderna, permitindo que armazenemos e transmitamos grandes volumes de informação de forma eficiente. Um dos algoritmos mais elegantes e fundamentais para essa tarefa é o de Huffman, que cria um sistema de codificação otimizado baseado na frequência dos caracteres.

A Árvore de Huffman é uma ferramenta genial para compressão de dados. Seu principal objetivo é tornar arquivos de texto (ou qualquer tipo de dado com elementos repetidos) menores, sem perder nenhuma informação.

A grande sacada é a seguinte:

Em sistemas normais, como o ASCII, cada caractere ocupa o mesmo espaço (geralmente 8 bits). A letra 'a' ocupa 8 bits e a letra 'z' também, não importa quantas vezes elas apareçam no texto.

O algoritmo de Huffman acha isso um desperdício. Ele propõe uma abordagem mais inteligente:

- Letras que aparecem com muita frequência devem usar códigos binários curtos.
- Letras raras podem usar códigos mais longos.

Pense nisso como o código Morse: a letra 'E', muito comum em inglês, é representada por um único ponto (.), enquanto a letra 'Q', que é rara, usa um código bem mais longo (--.). O princípio é o mesmo!

Ao substituir as letras de um texto por esses novos códigos de tamanho variável, o tamanho final do arquivo diminui drasticamente. Por exemplo, na palavra "bananaaa", a letra 'a' (que aparece 5 vezes) ganha o código super curto 1, economizando muito espaço.

A estrutura da árvore garante uma propriedade essencial chamada "código de prefixo", que significa que nenhum código é o começo de outro. Isso é crucial para que, na hora de descomprimir o arquivo, o computador consiga ler a sequência de 0s e 1s e saber exatamente onde termina o código de uma letra e começa o da próxima, sem nenhuma confusão.

Portanto, a Árvore de Huffman cria um dicionário otimizado para compactar dados, tornando a representação de informações muito mais eficiente.

Passo a passo do algoritmo (contagem, criação de nós, construção, códigos)

A ideia central do algoritmo de Huffman é criar códigos de prefixo de tamanho variável. Isso significa que caracteres mais frequentes em um texto recebem códigos binários mais curtos, enquanto os mais raros recebem códigos mais longos. A regra de ouro é que nenhum código pode ser o prefixo de outro, o que garante uma decodificação sem ambiguidades.

Para construir a árvore que gera esses códigos, o algoritmo segue uma abordagem gulosa (greedy):

- **Contagem de Frequência:** Analisa o texto para contar quantas vezes cada caractere aparece.
- **Criação de Nós:** Cada caractere único se torna um nó "folha" de uma árvore.
- **Construção da Árvore:** Utilizando uma fila de prioridades (min-heap), os dois nós com as menores frequências são repetidamente combinados para formar um novo nó interno. A frequência desse novo nó é a soma das frequências de seus filhos. Esse processo continua até que reste apenas um nó: a raiz da árvore.
- **Geração dos Códigos:** Ao percorrer a árvore a partir da raiz, atribui-se '0' para cada descida à esquerda e '1' para cada descida à direita. O caminho da raiz até uma folha forma o código Huffman para o caractere contido nela.

Exemplo: "banana"

O que o meu código faz?

Imagine que você deu ao programa a palavra "bananaaa". O programa executa as seguintes tarefas, em ordem:

1. **Contagem:** Primeiro, ele age como um detetive e inspeciona a palavra. Ele conta quantas vezes cada letra aparece. No final, ele sabe que há:
 - 'a': 5 vezes
 - 'n': 2 vezes
 - 'b': 1 vez
2. **Organização Inicial:** Ele cria pequenas "etiquetas" (ou nós) para cada letra única, anotando a sua frequência. Agora ele tem três etiquetas: (b, 1), (n, 2) e (a, 5).
3. **Construção da Árvore (A parte principal):**
 - Ele olha para as etiquetas e vê que (b, 1) e (n, 2) são as menos frequentes.
 - Ele as une, criando um novo "galho" com uma frequência somada ($1 + 2 = 3$).
 - Agora ele tem apenas duas coisas para comparar: a etiqueta (a, 5) e o novo galho que vale 3.
 - Seguindo a regra, ele junta esses dois, pois são os únicos que restaram. A árvore está pronta!
4. **Criação dos Códigos:** Com a árvore montada, ele percorre os galhos para criar os códigos binários. Ele segue outra regra simples:
 - Se for para a esquerda em um galho, anote '0'.
 - Se for para a direita, anote '1'.
 - Ele faz isso para cada letra, traçando o caminho do topo da árvore (a raiz) até a etiqueta da letra. O caminho que ele percorreu se torna o código daquela letra.
5. **Exibição do Resultado:** No final, o programa mostra uma "tabela de tradução" que ele criou, associando cada letra ao seu novo código binário. Para "bananaaa", o resultado seria algo como:
 - a: 1
 - n: 01
 - b: 00

Em resumo: o meu código lê um texto, conta a frequência das letras, constrói uma árvore otimizada com base nessa contagem e, por fim, usa essa árvore para gerar um novo código binário para cada letra.

Aplicações e vantagens

O algoritmo de Huffman tem diversas aplicações práticas:

- Compressão de texto.
- Transmissão de dados em redes.
- Formatos populares como JPEG, MP3, ZIP e outros métodos de compactação.

Entre as vantagens, destacam-se:

- Compressão eficiente sem perda de informação.
- Simplicidade de implementação.
- Adaptação automática ao conteúdo analisado.

9. PDL – Árvore B

- Introdução e definição

-Introdução

As Árvores B são estruturas de dados em forma de árvore balanceada, utilizadas principalmente em sistemas que lidam com grande volume de dados em memória secundária (como bancos de dados e sistemas de arquivos).

Foram introduzidas em 1972 por Rudolf Bayer e Edward McCreight.

O que é uma Árvore de Busca?

Uma árvore de busca é uma estrutura hierárquica que armazena elementos de forma ordenada.

Características gerais:

Cada nó possui múltiplas chaves (não apenas uma).

Os valores em cada nó estão ordenados.

Os “filhos de um nó” são organizados de acordo com os intervalos das chaves armazenadas.

-Definição de Árvore B

Uma Árvore B de ordem m ($m \geq 3$) obedece às seguintes regras:

Cada nó pode conter até $m-1$ chaves.

Cada nó (exceto raiz) deve ter pelo menos $\lceil m/2 \rceil$ filhos.

Todas as folhas estão no **mesmo nível** (balanceada por natureza).

As chaves em cada nó dividem o espaço de valores de forma que os filhos da esquerda contenham valores menores e os da direita, maiores.

- Regras de uma Árvore B

- Operações: inserção, busca e remoção

Inserção

Algoritmo inserir (chave k , Árvore B T):

1. Procurar o nó folha adequado para inserir k .
2. Inserir k na posição correta, mantendo a ordenação.
3. Se o nó exceder a capacidade ($m-1$ chaves):

Dividir o nó em dois.

Promover a chave do meio para o nó pai.

Repetir se necessário até a raiz.

...

Busca

Algoritmo Buscar(chave k, nó N):

1. Procurar em N a posição de k.
2. Se k estiver presente → sucesso.
3. Se N for folha → falha.
4. Caso contrário, seguir para o filho apropriado e repetir.

Remoção

Algoritmo Remover(chave k, Árvore B T):

- 1 Se k estiver em uma folha, remover diretamente.
 - 2 Se k estiver em nó interno:
 - 3 Substituir por predecessor ou sucessor.
 - 4 Remover do nó folha correspondente.
3. Se após remoção algum nó ficar com menos de $\lceil m/2 \rceil$ chaves:
Tentar redistribuir chaves com nó irmão.

Se não for possível, fundir nós.

- Vantagens, desvantagens, aplicações

Vantagens da Árvore B

Mantém altura $O(\log n)$, mesmo com grande quantidade de dados.

Ideal para armazenamento em disco (minimiza acessos).

Balanceamento automático sem necessidade de rotações complexas.

Desvantagens da Árvore B

Estrutura e código de implementação mais complexos.

Operações de divisão/fusão de nós podem ser custosas.

Menos eficiente que outras árvores (como AVL) em memória principal.

Aplicações da Árvore B

Sistemas de bancos de dados (Índices em disco).

Sistemas de arquivos (NTFS, EXT, HFS+).

- Conclusão

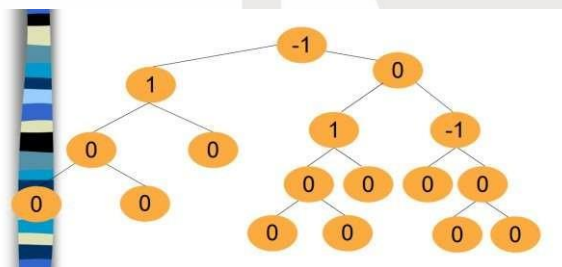
Por ser sempre balanceada e otimizada para leitura em blocos, é preferida em contextos de bancos de dados e sistemas operacionais.

- Conceito de balanceamento

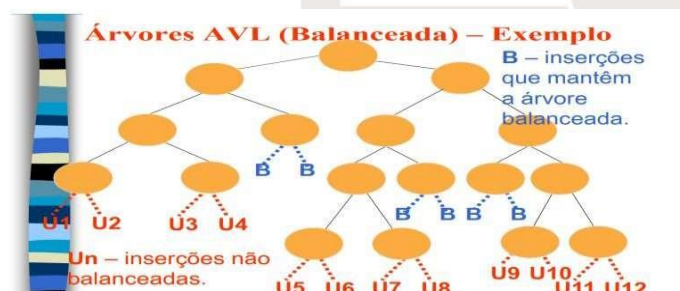
- Definição de árvore AVL (Altura máxima diferença 1)

O custo das operações de buscas, inserção, remoção e arrumação da estrutura da árvore mantém-se em $O(\log)$. Nomeada em homenagem aos seus criadores, Adelson-Velsky e Landis.

Cada nó tem balanceamento de 1, -1 ou 0. Se o valor do balanceamento for diferente de 1, -1 ou 0. Essa árvore não é balanceada.



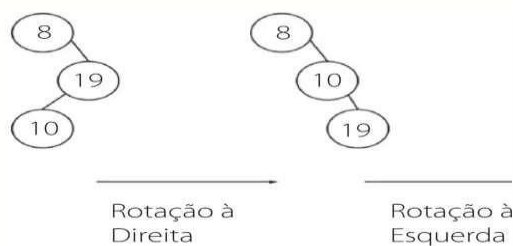
As inserções também podem deixar ou remover o balanceamento. **Exemplo:**



A transformação a ser feita na árvore para se manter balanceada é chamada de rotação. As rotações devem ser feitas quando existe um nó que está com FB fora do limite permitido. Ao inserir um elemento na árvore é preciso recalculcular o FB.

Rotação ocorre quando inserimos em nó da à direita para esquerda e da esquerda para direita.

1 - Exemplo:



2 – Exemplo:



11. Conclusão

O estudo das árvores em Estruturas de Dados revela-se fundamental para compreender como a Ciência da Computação organiza e processa informações de maneira eficiente. Ao longo deste trabalho, analisamos diferentes tipos de árvores, desde as mais simples, como a **Árvore Binária**, até estruturas mais sofisticadas, como as **Árvores AVL**, **Rubro-Negras**, **B**, **Digitais (Tries)** e **Huffman**.

A **Árvore Binária** apresentou-se como a base conceitual para as demais, fornecendo os princípios da hierarquia, da relação entre nós e do balanceamento básico. Apesar de sua simplicidade, seu desempenho pode se degradar caso não haja equilíbrio entre os ramos, tornando-a pouco eficiente em aplicações que exigem busca rápida em grandes volumes de dados.

As **Árvores AVL** surgem como solução a esse problema, garantindo que a diferença de altura entre as subárvores nunca ultrapasse um nível. Seu balanceamento automático por meio de rotações mantém a busca, inserção e remoção em tempo $O(\log n)$. Contudo, esse benefício tem um custo: a inserção e a exclusão são mais custosas do que em árvores simples, já que exigem ajustes constantes.

As **Árvores Rubro-Negras**, por sua vez, adotam um mecanismo de coloração para manter o equilíbrio. Diferentemente das AVL, permitem um balanceamento menos rígido, mas com operações de inserção e remoção mais rápidas, sendo amplamente utilizadas em compiladores e sistemas de banco de dados. Essa flexibilidade faz com que, na prática, muitas bibliotecas padrão das linguagens de programação utilizem Rubro-Negras para implementar estruturas como mapas e conjuntos ordenados.

Outro tipo de estrutura essencial são as **Árvores B**, que desempenham papel central em bancos de dados e sistemas de arquivos. Sua principal vantagem é a eficiência em acesso a dados armazenados em memória secundária, pois minimizam a quantidade de leituras em disco. Por isso, permanecem como uma das árvores mais utilizadas em sistemas de larga escala.

As **Árvores Digitais (Tries)** destacam-se no processamento de cadeias de caracteres, permitindo buscas rápidas por prefixos, dicionários e sistemas de autocompletar. Embora consumam mais memória, sua aplicação em áreas como compressão e roteamento de redes mostra sua relevância em cenários práticos.

Já a **Árvore de Huffman** mostrou como estruturas de dados podem ser aplicadas diretamente à compressão de informações. Baseada em frequências de ocorrência, essa árvore gera códigos binários de tamanho variável que reduzem significativamente o espaço necessário para armazenar dados, sendo utilizada em formatos de compactação como JPEG e MP3.

Por fim, o conceito de **Árvores Balanceadas** reforça a ideia central de que manter a estrutura organizada é essencial para garantir desempenho. Seja através de rotações, coloração ou divisão de nós, o balanceamento sempre aparece como solução para evitar a degradação do tempo de execução.

Em síntese, cada tipo de árvore atende a necessidades específicas: algumas priorizam velocidade de busca, outras eficiência em inserção/remoção ou economia de espaço. O estudo comparativo evidencia que não existe uma única estrutura perfeita, mas sim a escolha adequada conforme o problema a ser resolvido. Esse conhecimento é fundamental para a formação de engenheiros de software, pois fornece a base para projetar soluções computacionais mais rápidas, seguras e escaláveis.

12. Quadro Comparativo

13. Ao longo do trabalho, foram estudadas diferentes estruturas de árvores, cada uma com características próprias, vantagens e desvantagens. No entanto, para compreender melhor suas diferenças e identificar em quais contextos são mais adequadas, é fundamental realizar uma comparação sistemática.
14. A comparação permite visualizar de forma clara como essas estruturas se comportam em termos de **complexidade de operações, eficiência no balanceamento, consumo de memória e aplicações práticas**. Enquanto algumas árvores priorizam velocidade de busca, outras são mais vantajosas em sistemas de armazenamento de dados em larga escala ou em algoritmos de compressão.
15. Dessa forma, a seguir é apresentado um quadro comparativo que sintetiza as principais características das árvores abordadas neste estudo, oferecendo uma visão geral de suas

utilizações. Esse quadro busca destacar não apenas as vantagens e limitações de cada estrutura, mas também os cenários em que sua aplicação se torna mais adequada.

Quadro 1 – Comparação entre os principais tipos de árvores

Estrutura	Vantagens	Desvantagens	Aplicações principais
Árvore Binária	Simples de implementar, base conceitual para outras árvores	Pode ficar desbalanceada, degradando para $O(n)$	Introdução ao estudo de hierarquias
Árvore AVL	Busca, inserção e remoção em $O(\log n)$; sempre balanceada	Inserção e remoção mais custosas devido às rotações	Bancos de dados menores, compiladores
Rubro-Negra	Inserções e remoções rápidas; rebalanceamento mais simples que AVL	Menos eficiente em buscas do que AVL	Estruturas de dados em bibliotecas padrão
Árvore B	Ideal para grandes volumes de dados em disco; balanceada por natureza	Implementação mais complexa; operações de divisão/fusão custosas	Bancos de dados, sistemas de arquivos
Trie (Digital)	Busca eficiente por prefixos; útil para strings	Alto consumo de memória para alfabetos grandes	Autocompletar, dicionários, roteamento de IP
Huffman	Compressão eficiente; códigos de prefixo garantem decodificação correta	Específica para compressão, não serve para buscas gerais	Compactação de dados (ZIP, JPEG, MP3, etc.)

Fonte: Elaboração própria (2025).

13. Referências (ABNT NBR 6023:2018)

BUENO, Gabriel. Árvore Binária. REA-AED, [s. l.], [s. d.]. Disponível em:
https://gabrielbueno072.github.io/rea-aed/aula_arv.html

BAYER, Rudolf. Symmetric Binary B-Trees: Data Structure and Maintenance Algorithms. Acta Informatica, v. 1, n. 4, p. 290–306, 1972. Disponível em:
<https://dl.acm.org/doi/10.1007/BF00289509>

GUIBAS, Leonidas J.; SEDGEWICK, Robert. A Dichromatic Framework for Balanced Trees. In: Conference Record of the 19th Annual Symposium on Foundations of Computer Science. IEEE, 1978. Disponível em:

<https://sedgewick.io/wp-content/themes/sedgewick/papers/1978Dichromatic.pdf>

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009.

FABIO, Luiz. Aula de Árvores AVL. Universidade Federal Fluminense (UFF), [s. l.], [s. d.]. Disponível em: <http://profs.ic.uff.br/~fabio/Aula-AVL-1.pdf>

SILVA, José; ALMEIDA, João. Balanceamento de Árvores. Scribd, [s. l.], 2020. Disponível em: <https://pt.scribd.com/document/824423142/LIVRO-ESTRUT-DADOS-4-3-Balanceamento-de-arvore>